Ministry of Education, Culture and Research of the Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

LABORATORY WORK No. 8

The Same Calls, Inside Your Own OS

Student: ANDREI ELCISCEV
Group: FAF-243
Checked by: Reiţman Patricia

Chişinău, 2026

1 Objective

The objective of this laboratory work is to study process management and inter-process communication in the xv6 operating system. The first part demonstrates the use of the `fork()`, `exec()`, and `wait()` system calls by executing a child process and replacing its program with another executable.

The second part demonstrates communication between a parent and a child process using pipes. The programs are implemented in the xv6 environment and executed inside the operating system compiled during the course.

2 Part 1: The `run` Program

2.1 Implementation

The first program demonstrates the three main steps of process execution: creating a child process, replacing the child's program, and waiting for the child to finish.

The parent process prints its process ID and calls `fork()`. The child then executes the `echo` program using `exec()`, while the parent waits for the child's termination using `wait()`.

Listing 1: Implementation of user/run.c

```c
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[])
{
    printf("parent: my pid is %d\n", getpid());

    int pid = fork();

    if(pid < 0){
        printf("fork failed\n");
        exit(1);
    }

    if(pid == 0){
        printf("child: my pid is %d\n", getpid());

        char *args[] = {
            "echo", "hello", "from", "the", "child", 0
        };

        exec("echo", args);

        printf("exec failed\n");
        exit(1);
    }

    int status;
    wait(&status);

```

```
31      printf("parent: child %d ended with status %d\n",
32             pid, status);
33
34      exit(0);
35  }
```

The program was added to the `UPROGS` list in the xv6 `Makefile`:

```
1  $U/_run\
```

After recompiling and starting xv6, the program was executed from the xv6 shell using the command `run`.

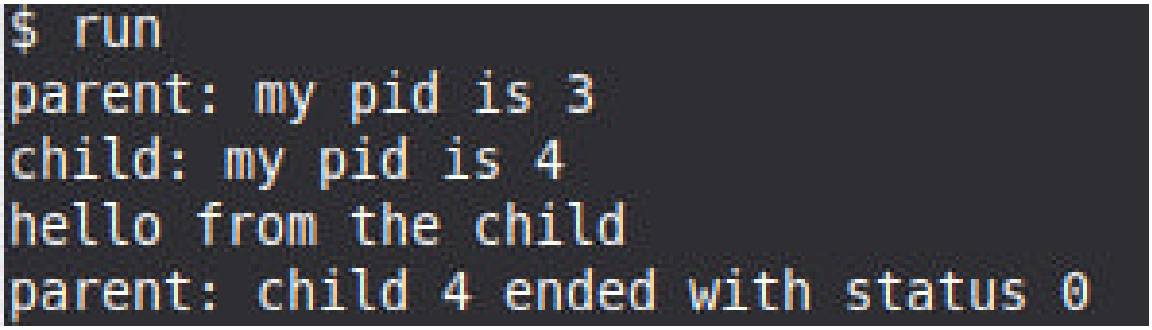


Figure 1: Execution of the run program with echo inside xv6

2.2 Executing the `ls` Program

The next step was to modify the child process so that it executes `ls` instead of `echo`. The argument array and the `exec()` call were changed as follows:

Listing 2: Executing ls in the child process

```
1  char *args[] = { "ls", 0 };
2  exec("ls", args);
```

The modified program was recompiled and executed. Instead of printing the message passed to `echo`, the child process now executes `ls` and displays the contents of the current directory.

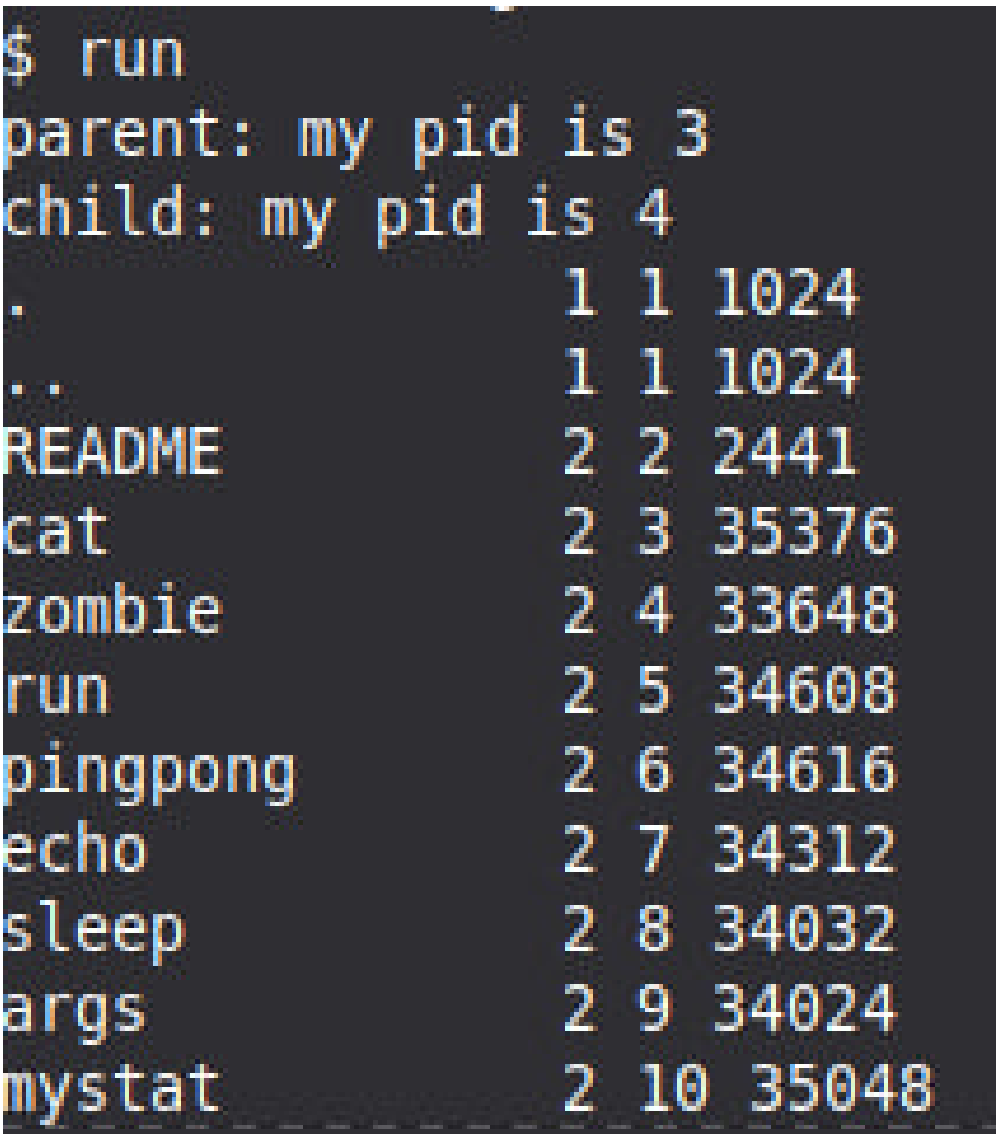


Figure 2: Execution of the ls variant inside xv6

2.3 Testing a Failed `exec()` Call

The final test in Part 1 involved attempting to execute a program that does not exist. The child process was modified to call `exec()` with the name `nosuchprog`.

Listing 3: Testing an unsuccessful exec call

```
1 char *args[] = { "nosuchprog", 0 };
2 exec("nosuchprog", args);
3
4 printf("exec failed\n");
5 exit(1);
```

When `exec()` fails, it returns to the original program instead of replacing the process image. Therefore, the next statement prints `exec failed`, and the child terminates with status 1.

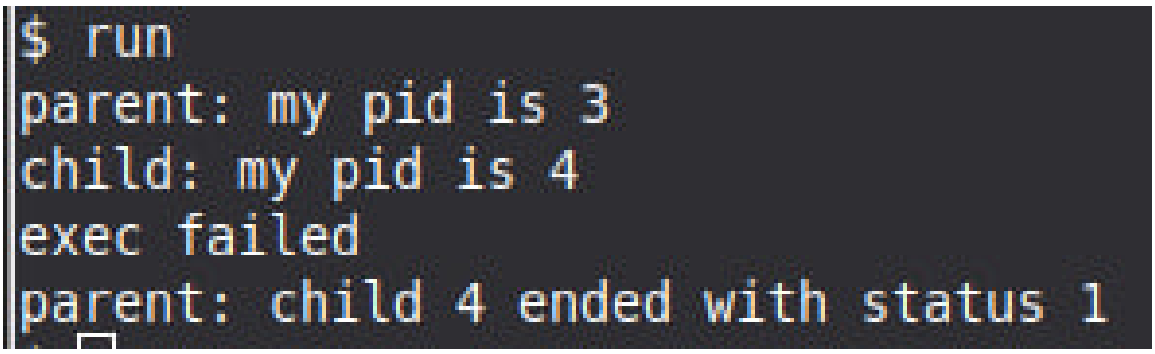


Figure 3: Execution of the nonexistent-program variant

2.4 Observe Questions

1. **Write the three calls that run.c uses in xv6 and the three calls run.c used on Linux in Lab 7. What is the same?**

   The three main calls in xv6 are `fork()`, `exec()`, and `wait()`. In Linux, the corresponding calls are typically `fork()`, `execvp()` (or another function from the `exec` family), and `wait()` or `waitpid()`.

   Their purposes are the same in both operating systems. The `fork()` call creates a child process, `exec()` replaces the current process image with another program, and `wait()` allows the parent to wait for the child to terminate. The main differences are the available interfaces and the implementation details of each operating system.

2. **Who prints "hello from the child": the code of run, or echo?**

   The `echo` program prints "hello from the child". When `exec()` succeeds, the child process begins executing the `echo` program, replacing the original program code. Consequently, the remaining statements in the child branch of `run.c` are not executed unless `exec()` fails.

3. **Why is the status 1 here and 127 on Linux?**

   In xv6, the program explicitly calls `exit(1)` after the failed `exec()` call. Therefore, the child terminates with status 1.

   In the Linux version, the program uses `exit(127)` after a failed `exec()` call. The difference comes from the explicit exit values chosen by the two programs. The failed `exec()` call itself does not automatically assign these exit statuses.

3 Part 2: The `pingpong` Program

3.1 Implementation

The purpose of the `pingpong` program is to demonstrate inter-process communication through pipes. The parent process sends one byte to the child through the first pipe. After receiving the byte, the child prints its process ID and the message `received ping`. It then sends the byte back through a second pipe.

The parent reads the returned byte, prints `received pong`, and waits for the child to terminate.

Two pipes are required because each pipe provides a unidirectional communication channel. The first pipe is used for communication from the parent to the child, while the second is used for communication from the child to the parent.

Listing 4: Implementation of user/pingpong.c

```c
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[])
{
    int p2c[2], c2p[2];
    char b = 'x';

    pipe(p2c);
    pipe(c2p);

    if(fork() == 0){
        close(p2c[1]);
        close(c2p[0]);

        read(p2c[0], &b, 1);

        printf("%d: received ping\n", getpid());

        write(c2p[1], &b, 1);

        exit(0);
    }

    close(p2c[0]);
    close(c2p[1]);

    write(p2c[1], &b, 1);

    read(c2p[0], &b, 1);

    printf("%d: received pong\n", getpid());

    wait(0);

    exit(0);
}
```

The variable `b` is initialized to a byte so that the parent can send a valid byte through the first pipe. The child reads this byte, sends it back through the second pipe, and then terminates.

The program was added to the `UPROGS` list in the xv6 `Makefile`:

```
$U/_pingpong\
```

After recompiling and starting xv6, the program was executed using the command `pingpong`.

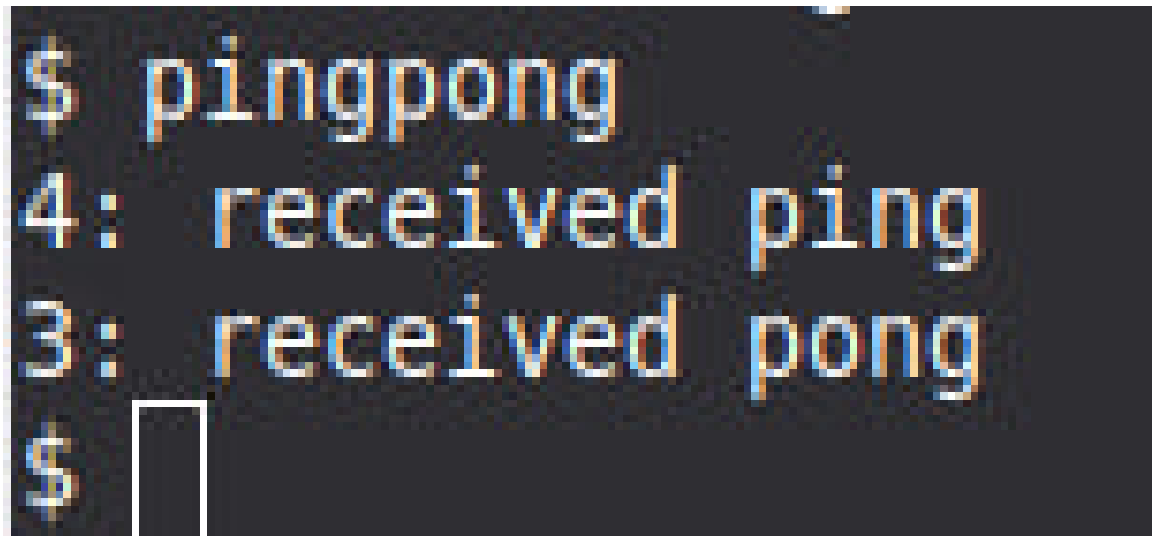


Figure 4: Execution of pingpong with ping printed before pong

3.2 Observe Questions

1. **Why do you need TWO pipes, and not one for both directions?**

   Two pipes are needed because a pipe is a unidirectional communication channel. The first pipe transfers data from the parent to the child, while the second transfers data from the child back to the parent.

   Using two separate pipes makes the direction of communication explicit and allows each process to close the pipe ends it does not need.

2. **Nothing in your code says "wait until the child has replied". Which call makes the parent wait?**

   The parent's call to `read(c2p[0], &b, 1)` makes the parent wait for the reply. If no byte is available and the pipe is still open, the read operation blocks until data becomes available or the pipe reaches end-of-file.

   The child writes the byte into the second pipe using `write(c2p[1], &b, 1)`, allowing the parent's read operation to complete.

   The later call `wait(0)` serves a different purpose: it waits for the child process to terminate. It does not receive the reply byte.

4 Conclusion

During this laboratory work, process management and inter-process communication were explored using the xv6 operating system.

The `run` program demonstrated the use of `fork()`, `exec()`, and `wait()`. The experiments with `echo`, `ls`, and a nonexistent program showed how a child process can execute another program and how a failed `exec()` call allows the original code to continue.

The `pingpong` program demonstrated communication between a parent and a child process through two pipes. The use of `read()` and `write()` showed how processes can exchange data and coordinate their execution. The experiment also illustrated the difference

between waiting for data through a blocking read and waiting for a process to terminate using `wait()`.

Overall, the laboratory work provided practical experience with fundamental operating-system concepts and demonstrated that xv6 uses the same basic process-management principles as Linux, despite differences in system-call interfaces and the available runtime environment.